Energy-Efficient Code via LLMs

Group 18

June 2026

Contents

1 Introduction & Context

1.1 Context & Background

Over the past few years, software engineering suffered a drastic change when it comes to the way that software is being produced and how fast it is being produced. One of the main factors that contributed to this drastic changed is represented by the emergence of Large Language Models (LLMs). Large Language Models were at first seen only as a tool that software developers used to replace traditional ways of learning like a simple search on the internet (e.g. StackOverflow), that would end up taking hours of their time. Nowadays, they became actual coding assistants, that sometimes generate a large part of the code that goes into production, making software a lot more efficient to produce, as software engineers only need to review the code that would have otherwise be written by themselves in a much longer time. Despite all these advantages, we have to raise the question that is being raised in every other major field, which is how sustainable the code that we are producing with these LLMs is.

1.2 Problem Statement

Despite the high amount of research that evaluates LLM-generated code, most of it performs this evaluation in terms of its correctness. This is, of course, a necessity when it comes to deciding whether or not to start using LLMs when producing software. However, we also need to look at how energy efficient a correct solution generated by an LLM is, and to see if LLMs are able to give us correct solutions that not only are ready to be released into production, but are also meeting the criteria for an energy-efficient solution.

2 Research Questions & Methodology

In this section, we analyze each of the paper's research questions (RQs), contextualize and explain the relevance of each question, and detail the steps and design choices the authors implemented to answer them. We also propose a non-trivial alternative method that the authors could have used, and we explain why choosing between the original and this alternative represents a real decision dilemma. To support our alternative, we cite existing literature.

Student Luca Section: Research Question 1

- **2.1.A Isolated Research Question (RQ):** "How does the energy efficiency of LLM-generated code vary when using different LLMs?" [1] - RQ_0
- **2.1.B Articulation & Relevance:** Our main objective is to find out how energy efficient is the code written by LLMs, by comparing LLM-generated code with human-generated code and code generated by a Green software expert. However, it is important to also do a comparison between the LLM models themselves, before transitioning to comparison with a human or a Green software expert. Answering this question will help us with setting the context of the experiment, to see if there is any energy consumption difference between the code generated by different models. This helps with seeing if the LLMs can be seen as a general, singular subject for comparison - meaning that all the models produce the same results in terms of energy consumption -, or if each of them produce different results; because after all, each LLM model is structured and trained differently.

  From a **practical** standpoint, this is important especially for the developers community, which nowadays, are increasingly using LLMs for writing code not just as a tool, but as an assistant. We also aim to compare the energy efficiency of the LLM-generated code on multiple hardware platforms: a server, a PC, and a Raspberry Pi. Based on the results we

will have, a developer can make an optimal choice in terms of energy efficiency when using an LLM as a coding assistant, depending on the hardware context in which he is working.

From a **theoretical** standpoint, we want to cover a less explored attribute for evaluating the quality of LLM-generated code. So far, evaluations of code generated this way consisted of only checking the correctness, i.e., the code produces the right output for a given problem. With the analysis of energy consumption, we now can think of a new way to evaluate LLMs. Moreover, we will also obtain a better perspective over how different models, having different architectures, training methods, training datasets and number of parameteres perform with respect to this evluation of energy efficiency. Maybe non of them were specifically trained with the purpose of generating energy efficient code in mind, but somehow, based on the different properties mentioned earlier, they might have "inherited" this capacity.

- **2.1.C Methodological Deconstruction:** We aim to select the top 6 accessible, unique LLMs in the difficult category of the EvoVal leaderboard, as of July 2024, based on two main criteria:
  Accessbility: the model is either open-source of available through API.
  Uniqueness: the modes uses a different base model compared to the other already-chosen LLMs.

  This way of choice will give us a certain level of external validity, as we will have a larger variety of models in terms of architecture, sizes, and training method.

  Moving forward to the coding problem selection, we want to select 100 coding problems from the Difficult category in EvoVal, since we want to generate Python code with higher computational complexity and additional requirements compared to ther other categories. We will only keep the problems for which each of the chosen LLMs generated a correct solution, since correctness will need to be already validated. We will evaluate these solutions based on the test cases we will extract from EvoVal. These initial solutions that we will test in order to see if they will provide the correct results will be the "functional solutions", representing the solutions that we will extract based on the original prompt given to each LLM, for each problem, from EvoVal.

  These original prompts that will create the functional solutions will also used to create different type of prompts. As this is not highly relevant for this specific research question, we won't dive deeper into what these different prompts represented, but we mention the fact that they will have the purpose of generating code specific to different criteria mentioned in each prompt. If a solution generated from an LLM, for a specific prompt, will prove to not be correct after 6 reprompts, that specific solution will be removed from the analysis. Once again, the solutions will be verified against the extracted test cases from EvoVal to see if they will be correct. All the remaining solutions will be executed on all three hardware platorms that we chose, with the purpose of strengthening the external validity by offering results accross a wide spectrum of hardware specifications:
  Server: Intel Xeon Silver 4208, 384GB RAM, Ubuntu 20.04.
  PC: Intel Core i9, Nvidia RTX 4070, 64GB RAM, Ubuntu 24.04.
  Raspberry Pi: (Cortex-A72 ARM v8, 8GB RAM, Raspberry Pi OS.

  We will calculate a number X_p^m (m - hardware machine, p - problem) of iterations with the purpose of achieving a runtime of 3 minutes. On each machine m, all test cases will be executed for each variant of a solution to problem p for X_p^m iterations. Each of these runs will be repeated 21 times on each hardware platform.

- **2.1.D Alternative Methodology Contrast:**
  - **The Dilemma:** Instead of generating and exeucting code in a controlled experiment, one good alternative could be to gather code from software repositories, and apply the evaluation on that. This would imply collecting large sampels of real-world Python code generated by each of the chosen LLMs, from publicly available sources (for example, GitHub repositories). Instead of executing the code the energy efficiency analysis could be done based on energy-costly code patterns (like nested loops, redundant operations, or inefficient data structured, some of which are also mentioned in the original paper). Deciding between the original methodology and this one could represent a dilemma because of the following facts:
    1. The original methodology uses benchmark problems from EvoVal in a controlled environment. This can raise the question of how much the results would be actually applicable in the real world. This new approach would use code that developers actually developed in production, giving the result more validity. However, the experiment control is sacrificed, as using publicly available code could introduce confounding factors, like post-editing of LLM output (by the developer), or maybe false attributions of which LLM generated which code.
    2. The static analysis employed in the new method allows for much more freedom in terms of time. Quantity-wise, a lot more solutions could be evaluated per LLM, however, accuracy is sacrificed compared to the original methodology.
    3. A much wider range of LLMs could become available by collecting data from publicly available sources. And not just the model variety would increase, but also the variety of programming contexts. However, the possibility of also considering hardware platforms would be sacrificed, which was proved to be quite important in the findings of the paper.
  - **Literature Support:** Literature support can be found at [2].

Student Hugo Section: Research Question 2

- **2.2.A Isolated Research Question (RQ):** "What is the difference between human code and LLM-generated code in terms of energy-efficiency?" [1] - RQ_1
- **2.2.B Articulation & Relevance:** In software creation, tools that use LLMs to generate code are used more often by new and experienced developers. Before this paper, research was mainly focussed on performance, readability, and maintainability of LLM generated code. Energy-efficiency has received (far less) attention in prior research, while software sustainability and Green AI are becoming more important in research and industry. If LLM-generated code is systematically less energy efficient than code by human developers, then widespread adaption of tools using these LLMs leads to large-scale energy waste. Before being able to investigate RQ_2 and RQ_3, we first must understand how LLM-generated code performs relative to human written code (RQ_1), as a baseline.
- **2.2.C Methodological Deconstruction:** Quantitative, empirical research design structured as a controlled experiment. Study design follows guidelines for empirical software engineering [3], [4] and energy efficiency assessment [5], [6].

  Independent variable: code producer (human or LLM). Dependent variable: energy consuption. Energy consumption can differ between machines, therefore doing experiments on server, pc, raspberry. Canonical solutions: human written solutions in EvoEval benchmark. Functional solutions: six selected LLMs generate solutions using the original EvoEval prompt, no prompt modification w.r.t. energy effiency. Solution variant is extecuted 21 times on each platform, to factor out random occurrences. EnergiBridge used

n server and PC to measure energy consumption. Monsoon Power Monitor on the Raspberry Pi.

- **2.2.D Alternative Methodology Contrast:**
  - **The Dilemma:** The canonical solutions from EvoEval is only a single implementation per problem. The details of the writer and the environment when it was produced are unknown. Human side of comparison is therefore small and maybe unrepresentative. Alternative is to recruit multiple developers and let them independently solve the same problem. Varying in experience, coding style, and optimization skills to generalize more. Downside is that recruiting is very time consuming (as Justus Bogner stated in his guest lecture), adds confounding factors (program language proficiency differences, energy-efficiency awareness), which is avoided by canonical solutions. Dillema: canonical solutions has higher reproducability (you can take the exact same solutions in a retry) but lower representativeness. Recruiting developers reflects real world conditions better but has lower experimental control.
  - **Literature Support:** Literature support can be found at [7].

Student Silvio Section: Research Question 3

- **2.3.A Isolated Research Question (RQ):** "What is the impact of prompt engineering techniques on the energy efficiency of LLM-generated code?" [1] - RQ_2
- **2.3.B Articulation & Relevance:**
  Relevance: Prompt engineering is an emerging field, which tries to determine the optimal prompt structure to obtain LLM outputs tailored to certain requirements. At a deeper level the question is trying to establish whether it is worth it to invest time and resources in prompt engineering in an energy saving context.
  Results: From the results we can surmise that while prompt engineering does have an effect on the code generated by the LLM, this change in output code does not necessarily imply an improvement in energy efficiency, especially when the machine running the code varies. This presents quite a challenge as a developer would need to know a priori what hardware the software would run on. Depending on the problem at hand this might not be possible. The findings evident from this study also seem to agree with the limited theory that is slowly being established.
- **2.3.C Methodological Deconstruction:**
  Research structure: The research question and the following process follow a well-thought out plan; in line with the established computer science research pipeline, the inductive approach applied by the researchers is common in the software engineering field and is well suited for a niche and emerging field such as prompt engineering. The experiment choice is also well suited, as running in vivo experiments involving real-life like code bases would be outside the scope of a single papepr.
  Reproducibility: Regarding the reproducibility of the experiment, the author does a good job of describing the setup used, both in hardware and testing framework, the author does not provide executables which could be used to replicate the energy usage measurements. Moreover some of the LLMs used are queried through API and are thus susceptible to architecture changes and updates which makes recreating results impossible. Another issue with running benchmarks with LLMs is their inherently non-deterministic behaviour, the code generated by prompting is not necessarily going to be the same at each prompt run. Multiple runs can be executed and statistics can be extracted from the obtained distribution, but the high variance between outputs makes the experiment less valuable.

- **2.3.D Alternative Methodology Contrast:**
  - **The Dilemma:** Write dilemma here… Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
  - **Literature Support:** Write support here… Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Student Gabriel Section: Research Question 4

- **2.4.A Isolated Research Question (RQ):** "What is the difference between code developed by a Green software expert and LLM-generated code in terms of energy efficiency?" [1] - RQ_3
- **2.4.B Articulation & Relevance:** As more developers rely on AI tools to produce code faster, the volume of code that is generated and deployed is also likely to increase. This makes it important to evaluate code not only for functional correctness, but also for non-functional qualities such as energy efficiency. In many cases, developers focus primarily on functionality and may overlook sustainability concerns. Therefore, it is valuable to understand how LLM-generated code compares with code written by a Green software expert in terms of energy efficiency, since this can have significant implications for the sustainability of software applications and the environment.
- **2.4.C Methodological Deconstruction:** The authors utilized an Empirical Style, such that they first setup a controlled experimental environment to execute code across different hardware platforms. The proposition claims a relationship bewtween two constructs: code and energy efficiency. To test this proposition, the hypothesis is that the expert-written code consumes less energy than the LLM-generated code. The moderating variable is represented by three different hardware platforms: server, laptop, and Raspberry Pi. They selected a set 9 coding problems from the EvoEval benchmark, but focused on specifically 4 of these for the green software expert and LLM-generated code comparison. For AI generation, they used 6 different LLMs combined with 4 distinct prompting strategies. In regards to the green software expert, a specialist with over 7 years of experience manually optimized the code using the problem description, the canonical solutions, test cases and a set of specially curated guidelines. Finally, to measure the dependent variable, defined as energy consumption in Joules, the researchers executed each solution for approximately 3 minutes. They repeated these measurements 21 times on each hardware platform to account for intrinsic energy fluctuations and to strengthen the statistical power.
- **2.4.D Alternative Methodology Contrast:**
  - **The Dilemma:** An alternative method the authors could have used is a Taxonomical Style, where they could have focused on the structural analysis of the source code generated by both the Green software expert and the LLMs. So, instead of setting up an experimental environment to measure energy consumption, the code could be evaluated based on structural features that are known to influence energy efficiency, such as data structures, algorithmic complexity, and design patterns. The fundamental trade-off between the original method and this alternative is that the physical execution provide a direct measurement of energy consumption, which is a more concrete and empirical approach. In contrast, the taxonomical method relies on theoretical analysis. By classifying and labeling the code based on its structural features, the risk is that it may not capture the real-world energy consumption accurately.
  - **Literature Support:** Literature support can be found at [8].

Student Daniele Section: Research Question 5

- **2.5.A Isolated Research Question (RQ):** To what extent does the hardware platform[1] impacts the energy efficiency of code generated by LLMs? See footnotes[2].
- **2.5.B Articulation & Relevance:** When developers use LLMs to write code, we usually assume it will behave similarly no matter where it runs. However, different hardware architectures handle code instructions differently. This research question asks whether code that is energy efficient on a high-end server remains energy efficient on an everyday PC or a resource-constrained IoT device like a Raspberry Pi.

  Billions of lines of AI-generated code are being pushed into production environments globally. If an LLM generates a script that runs perfectly on a developer's local PC but suddenly wastes massive amounts of electricity when running on a remote server, it leads to huge financial costs and a larger environmental footprint.
- **2.5.C Methodological Deconstruction:** To answer this specific RQ, the researchers ran all tests on three physical machines with very different hardware specs. The machines were a high-end server, a PC, and a Raspberry Pi. To make sure they were measuring only the code's energy and not background noise, they turned off all non-essential background processes, set the CPU power governor to performance, and set a cooldown period of 60 seconds between tests so the machines wouldn't overheat and alter results. They executed each piece of code 21 times per machine (discarding the first run to avoid cold starts). Because the scripts run very quickly, they wrapped the functions in loops and subtracted the energy of an "empty pass" function to isolate the code's precise consumption. They recorded energy consumption using EnergiBridge for a total of approximately 4.6 billion energy measures. The authors ran a statistical test called an Two-Way Aligned Rank Transformation (ART) ANOVA. This specific test allowed them to see if the interaction between the Model used and the Hardware platform was statistically significant. The researchers also tried to nudge LLMs into a more appropriate solution for the platform by telling the AI the hardware profile the code was going to be executed on, but as [1] - RQ_2 shows, prompt engineering does not have a significant impact on energy consumption, sometimes even getting worse results.
- **2.5.D Alternative Methodology Contrast:**
  - **The Dilemma:** Instead of measuring physical power usage on real machines, the authors could have used profiling tools like `perf` to track micro-architectural metrics like cache misses and branch mispredictions. These micro-architectural metrics could also be obtained by running the LLM-generated code inside specialized software simulators (like GEM5) that support different CPU instruction set architectures (e.g., x86 or ARM) and configurable memory. Testing on real hardware gives you actual, authentic physical data reflecting real-world usage. However, it suffers from a lot of unpredictable hardware noise. It is also incredibly slow and expensive to test and does not generalize well to future hardware. On the other hand, the proposed alternative completely isolates the software. It can tell you exactly why a piece of code is draining power by tracking microscopic variables (e.g., "this nested loop caused 5,000 L1 cache misses on ARM but only 200 on x86"). It is 100% repeatable with zero environmental noise. The downside is that the results are software approximations, and they do not capture the chaotic nature of real-world systems.

[1] Server, PC, and Raspberry Pi

[2] Even tho this is not an explicit research question of the paper, the impact of the hardware platform is a recurring theme of this research.

- **Literature Support:** In [9], on their research on software energy consumption, instead of just tracking raw system power, they utilized micro-architectural profiling metrics like context switches and OS thread overhead to explain the exact hardware mechanics causing code to use more energy.

3 Conclusion & Reference List

3.1 Conclusion

In this research, we looked at how energy-efficient LLMs are when generating code. We broke this down into five research questions, covering how AI-generated code compares to human-written code, different prompting strategies, green software expertise, and hardware platforms. Our findings show that while LLMs are good at writing code that works, writing code that's energy-efficient is still an unsolved problem. Energy consumption varies a lot between models, and human-written code, particularly from green software experts, still has a clear sustainability edge. Prompt engineering techniques meant to help were often inconsistent in practice. One key insight is that energy efficiency is tied to where the code actually runs. Code optimized for a desktop PC can behave very differently on a server or a small device like a Raspberry Pi, sometimes using far more power than expected. Ultimately, this research suggests that the software engineering community needs to think beyond development speed if we want a more sustainable digital future.

3.2 Reference List

[1] R. Apsan, V. Stoico, M. Albonico, R. Dhar, K. Vaidhyanathan, and I. Malavolta, "Generating Energy-Efficient Code via Large-Language Models – Where are we now?." [Online]. Available: https://arxiv.org/abs/2509.10099

[2] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, Hyderabad India: ACM, May 2014, pp. 92–101. doi: 10.1145/2597073.2597074.

[3] F. Shull, J. Singer, D. I. K. Sjøberg, and S. (Online Service, *Guide to Advanced Empirical Software Engineering*. London: Springer London, 2008.

[4] C. Wohlin, P. Runeson, HöstMartin, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in software engineering*. Berlin: Springer Berlin, 2014.

[5] A. Guldner *et al.*, "Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components—Green Software Measurement Model (GSMM)," *Future Generation Computer Systems*, vol. 155, pp. 402–418, 2024, doi: https://doi.org/10.1016/j.future.2024.01.033.

[6] J. Mancebo, F. García, and C. Calero, "A process for analysing the energy efficiency of software," *Information and Software Technology*, vol. 134, p. 106560, June 2021, doi: https://doi.org/10.1016/j.infsof.2021.106560.

[7] W. Wang, H. Ning, G. Zhang, L. Liu, and Y. Wang, "Rocks Coding, Not Development–A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks." [Online]. Available: https://arxiv.org/abs/2402.05650

[8] S. Hao, D. Li, W. G. J. Halfond, and R. Govindan, "Estimating mobile application energy consumption using program analysis," in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 92–101. doi: 10.1109/ICSE.2013.6606555.

[9] G. Pinto, F. Castor, and Y. D. Liu, "Understanding energy behaviors of thread management constructs," in *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*, Portland Oregon USA: ACM, Oct. 2014, pp. 345–360. doi: 10.1145/2660193.2660235.